Lab 3: Parallel Sieve of Eratosthenes

Adomas Vensas, student ID 2525104. Problem 4. Written in C# (.NET 8) with threads.

1. Problem

Find all prime numbers up to N with the Sieve of Eratosthenes: take 2 and cross out all its multiples, then take the next number that is not crossed out and do the same, and so on. The numbers that are left are primes. My program counts them.

2. How I parallelized it

- **Dividing the work:** first one thread finds the small primes up to √N (this is very fast). Then the numbers up to N are split into equal parts (segments). Each part can be sieved on its own using the small primes.
- **Giving parts to threads:** there is a shared counter that shows which part is next. When a thread is free, it takes the next part and increases the counter. So faster threads just take more parts.
- **Synchronization:** the counter and the total are changed with atomic `Interlocked` operations. The main thread waits for all threads with `Join`.

3. Experiment

I ran the program on the MIF cluster on one node with 2 × Intel Xeon Gold 6252 processors, using 24 cores. Each test was run 3 times and the best time was taken. Speedup = time with 1 thread / time with p threads.

Test 1. Diferent N (part size 262,144):

Threads	N = 10^7		N = 10^8		N = 10^9	
	Time, ms	Speedup	Time, ms	Speedup	Time, ms	Speedup
1	14.8	1.00	142.2	1.00	1439.3	1.00
2	7.8	1.90	70.8	2.01	722.8	1.99
4	6.2	2.39	37.9	3.75	385.8	3.73
8	4.5	3.27	21.5	6.62	200.4	7.18
16	4.5	3.26	16.4	8.65	111.0	12.97
24	4.0	3.66	15.2	9.35	92.4	15.58

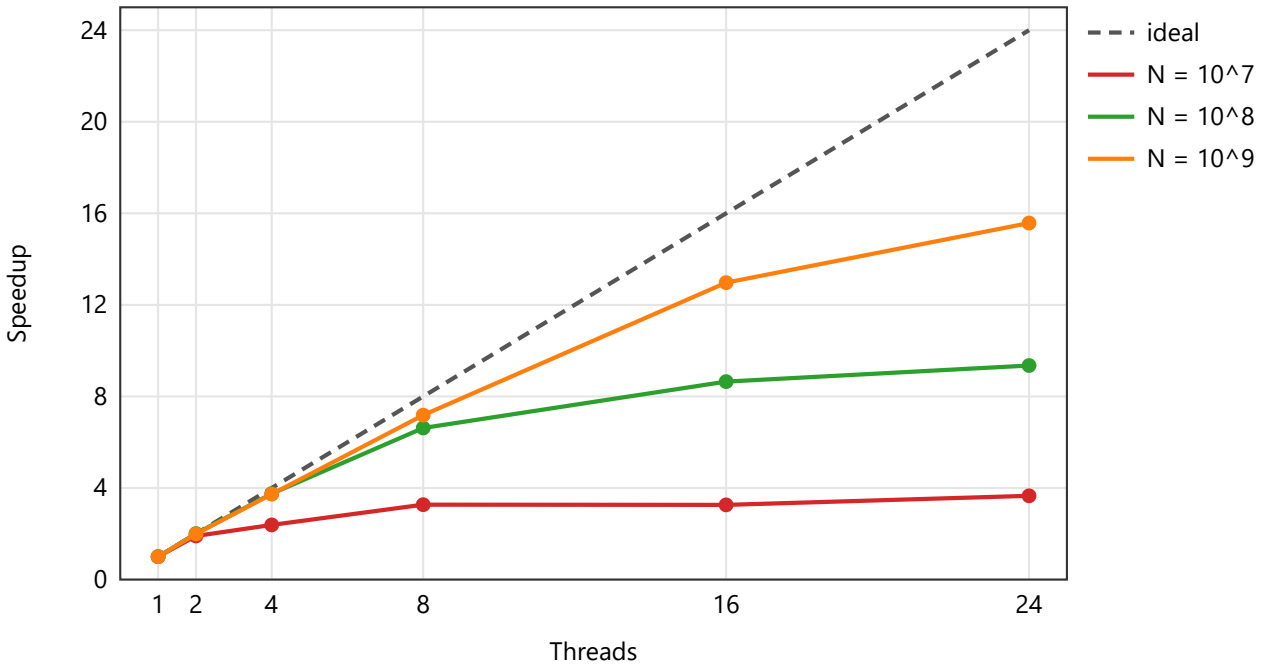


Figure 1. Speedup depending on number of threads. Dashed line = ideal speedup.

Test 2. Different part (segment) size, N = 10^9:

Part size	1 thread, ms	24 threads, ms	Speedup
4K	6696.5	390.0	17.17
16K	2530.3	142.3	17.78
64K	1632.1	109.4	14.92
256K	1438.3	95.0	15.14
1M	1634.8	100.1	16.33
4M	2274.3	374.3	6.08
16M	2692.9	1054.6	2.55

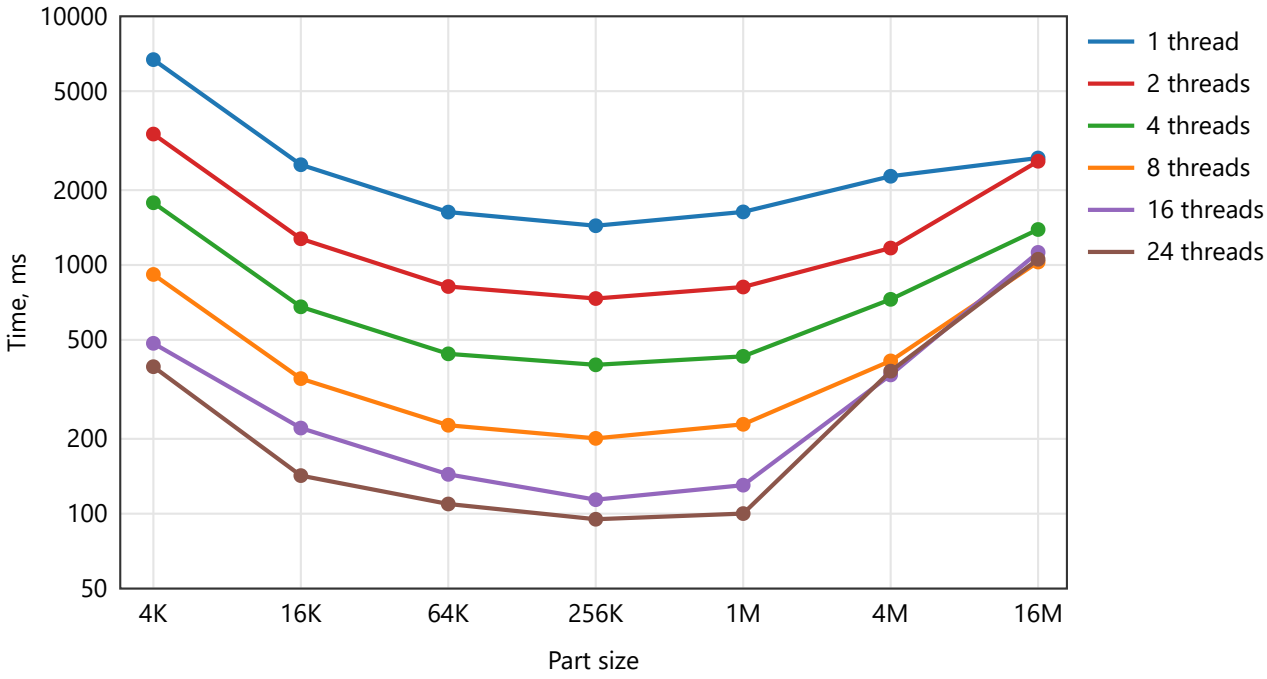


Figure 2. Time depending on part size, N = 10^9.

4. Conclusions

- **Maximum speedup:** 15.6 times with 24 threads (N = 10^9): from 1439 ms to 92 ms.
- **How good it is:** almost linear up to 8 threads, very good with 16 threads (13 times), good with 24 threads. For small N (10^7) the speedup is small (3.7 times), because the program finishes in a few milliseconds.
- **Why it is not perfect:** creating threads takes some time, at the end some threads wait for the last parts, all cores share the same memory, and the procesor runs a bit slower when all cores are busy.
- **Is it scalable:** yes, for big N. The bigger N is, the better the speedup (3.7, 9.4 and 15.6 with 24 threads). Part size also matters: 64K–1M works best. Too small parts add extra work, too big parts do not fit in the cache and there are too few of them to share between threads.